

앱프로그래밍 4주차 2차시 (실습)

To-do List 앱 만들기



인공지능학과 | 담당교수 안 효 정

목차

1	학습 목표	🎯	6	이벤트 처리 (onChange/onSubmit)	🖱️	11	확장 아이디어	🚀
2	지난 시간 복습	🔄	7	실습 단계별 구현	</>	12	앱 실행화면	📱
3	오늘의 실습 개요	☰	8	참신한 팁	💡	13	학습 정리	✓
4	컴포넌트 생성 및 구조	🏗️	9	전체 코드	📄	14	퀴즈 및 참고자료	?
5	상태와 입력/출력 관리	🗄️	10	CSS 샘플	🖋️			

학습 목표

-  이벤트(onChange, onSubmit) 실습 적용
폼 제어 및 사용자 입력 처리 방법 학습
-  To-do List 앱 기본 기능 구현
추가, 삭제, 상태 변경 기능 중심 실습
-  재미있고 실용적인 팁 학습
이모지, 우선순위, 자동저장(localStorage) 등

```
function TodoApp() {  
  const [todos, setTodos] = useState([]);  
  const [text, setText] = useState("");  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    setTodos([...todos, { text, done: false  
  }]);  
  }  
  ...  
}
```

지난 시간 복습



Controlled Form 개념

React 상태(state)로 완전히 제어되는 폼 요소



onChange: 입력값 → state

입력값이 변할 때마다 상태 업데이트



onSubmit: 제출 제어

e.preventDefault()로 기본 동작 방지



폼 검증 및 유효성 체크

사용자 입력 검증 및 에러 처리

```
function LoginForm() {  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  
  const handleSubmit = (e) => {  
    e.preventDefault(); // 기본 제출 방지  
    // 유효성 검사  
    if (!email || !password) {  
      alert("모든 필드를 입력해주세요");  
      return;  
    }  
    // 로그인 처리...  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input value={email} onChange={e =>  
        setEmail(e.target.value)} />  
      <input type="password" value={password}  
        onChange={e => setPassword(e.target.value)} />  
      <button type="submit">로그인</button>  
    </form>  
  );  
}
```

오늘의 실습 개요

-  **입력: 새 할 일 작성**
Controlled Form 패턴으로 입력값 관리
-  **추가: onSubmit으로 목록 반영**
폼 제출 이벤트로 할 일 등록 구현
-  **출력: map으로 리스트 렌더링**
배열 데이터를 UI 요소로 변환하기
-  **조작: 완료/삭제 기능 구현**
체크박스과 삭제 버튼으로 할 일 관리
-  **팁: 실용적 기능 추가**
우선순위, 이모지, 날짜, 자동저장 등

사용자 입력

```
onChange={(e) => setText(e.target.value)}
```

폼 제출 처리

```
onSubmit={handleSubmit}
```

상태 업데이트

```
setTodos([...todos, { text, done: false }])
```

리스트 렌더링

```
{todos.map((todo) => ...)}
```

사용자 조작 & 자동저장

```
useEffect(() => localStorage.setItem(...))
```

컴포넌트 구조 설계

-  TodoApp 컴포넌트 생성
src/components/ToDoApp.jsx 파일 생성
-  컴포넌트 함수 작성
export default function으로 함수형 컴포넌트 생성
-  기본 UI 구성요소
헤더, 입력 폼, 리스트 영역, 상태 표시 영역
-  App.js 연결
메인 App 컴포넌트에서 ToDoApp 임포트 및 사용

```
export default function ToDoApp() {  
  return (  
    <div>  
      <h2>To-do List</h2>  
      // 여기에 폼과 할 일 목록이 추가될 예정  
    </div>  
  );  
}
```

App.js



ToDoApp.jsx



UI 컴포넌트들
(폼, 리스트, 항목 등)

src/components/ToDoApp.jsx 생성 후 App.js에서 불러오기

상태(state) 준비

text 상태

입력창의 현재 값을 저장하는 상태

사용자 입력을 관리하고 제어된 컴포넌트(Controlled Component) 구현

todos 상태

할 일 목록을 배열 형태로 저장하는 상태

각 할 일은 id, text, done 속성을 가진 객체

상태는 컴포넌트가 다시 렌더링될 때마다 유지되며, 상태가 변경되면 React는 해당 컴포넌트를 자동으로 다시 렌더링합니다.

```
import { useState } from 'react';

function TodoApp() {
  // 입력값을 관리하는 상태
  const [text, setText] = useState("");

  // 할 일 목록을 관리하는 상태
  const [todos, setTodos] = useState([
    // 초기 데이터 예시
    { id: 1, text: "리액트 공부하기", done: false },
    { id: 2, text: "투두리스트 만들기", done: true }
  ]);

  // 이후 컴포넌트 내용...
}
```

입력값 관리 (onChange)



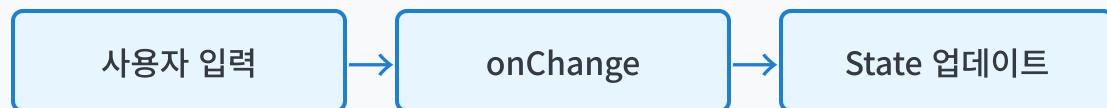
Controlled Input 패턴

React의 state가 "단일 진실 공급원(SSOT)"이 되어 입력 요소의 값을 완전히 제어



onChange 이벤트

입력값 변경 시마다 호출되어 state를 업데이트하고 화면을 재렌더링



```
const [text, setText] = useState("");

// Controlled Input 패턴

```


할 일 추가 (onSubmit)



form 태그 활용

HTML의 폼 제출 기능을 React에서 활용
접근성과 기본 키보드 제출(Enter) 지원



e.preventDefault()

기본 제출 동작(페이지 새로고침) 방지
Single Page Application 유지 필수



제출 버튼 구현

type="submit" 속성으로 제출 버튼 지정
form의 onSubmit 이벤트로 제출 처리

```
// 폼 태그와 onSubmit 속성 사용
<form onSubmit={handleSubmit}>
  <input
    value={text}
    onChange={e => setText(e.target.value)}
    placeholder="할 일을 입력하세요"
  />
  <button type="submit">추가</button>
</form>

// 제출 처리 함수
const handleSubmit = (e) => {
  e.preventDefault(); // 페이지 새로고침 방지
  // 할 일 목록에 새 항목 추가 로직
}
```

handleSubmit 함수 구현

1 빈 값 검증

trim()으로 공백 제거 후 검증하여 빈 문자열 입력 방지

2 새 항목 추가

불변성을 지키면서 새 객체를 todos 배열에 추가

3 입력창 초기화

추가 완료 후 입력값 초기화 (setText(""))

```
const handleSubmit = (e) => { // 기본 이벤트 방지
  e.preventDefault(); // 입력값 검증 - 빈 값 방지
  const trimmed = text.trim(); if (!trimmed) return; // 새 항목 객체 생성 및 배열에 추가
  setTodos([ ...todos, { id: Date.now(), text: trimmed, done: false } ]); // 입력창 초기화
  setText(""); };
```

목록 출력 (map)



map 함수란?

배열의 각 요소를 순회하며 새로운 배열을 반환하는 JavaScript 배열 메서드



React 리스트 렌더링

배열의 각 항목을 React 요소로 변환하여 UI에 표시



key prop 필수

각 리스트 항목은 고유한 key 속성을 가져야 함 (다음 슬라이드에서 상세 설명)

```
// map을 사용한 할 일 목록 렌더링
return ( <ul> {todos.map(todo => ( <li key=
{todo.id}> {todo.text} </li> ))} </ul> );
```

렌더링 결과:



리액트 공부하기



자바스크립트 문서 작성



운동하기

리스트 항목 key prop의 중요성

🔑 Key는 React의 요소 식별자

React는 key를 통해 어떤 항목을 변경, 추가, 삭제할지 식별합니다. 이를 통해 DOM 업데이트를 효율적으로 수행할 수 있습니다.

⚠️ key를 생략하면 발생하는 문제

key가 없으면 React는 항목의 위치(인덱스)에 의존하게 되어, 리스트 변경 시 불필요한 리렌더링이나 예상치 못한 UI 버그가 발생할 수 있습니다.

✅ Best Practice

항상 안정적이고 고유한 ID를 key로 사용하세요. 배열의 인덱스는 항목 순서가 변경될 경우 문제를 일으킬 수 있어 권장하지 않습니다.

주의: 인덱스를 key로 사용하면 항목 순서 변경, 추가, 삭제 시 문제가 발생할 수 있습니다.



잘못된 사용법

```
{todos.map((todo, index) => ( <li key={index}> {todo.text} </li> )))}
```



올바른 사용법

```
{todos.map(todo => ( <li key={todo.id}> {todo.text} </li> )))}
```

```
// 할 일 추가 시 고유 ID 생성 예시
const handleSubmit = () => {
  setTodos([...todos, {
    id: Date.now(), // 고유한 ID
    text: inputText,
    done: false
  }]);
}
```

완료 체크박스 추가



체크박스 UI 구현

할 일 항목마다 체크박스를 추가하여 완료 상태를 시각적으로 표시합니다.



이벤트 연결 (onChange)

체크박스 클릭 시 toggleDone 함수를 호출하여 해당 할 일의 완료 상태를 변경합니다.



상태 관리 흐름

체크박스 상태는 todos 배열의 done 속성으로 관리되며, 체크 시 렌더링에 자동 반영됩니다.



리액트 학습하기



To-do List 앱 만들기

```
// 각 할 일 항목에 체크박스 추가하기
<li key={t.id}>
  <input
    type="checkbox"
    checked={t.done}
    onChange={() => toggleDone(t.id)}
  />
  {t.text}
</li>

// 체크박스 클릭 시 실행되는 함수
const toggleDone = (id) => {
  setTodos(todos.map(t =>
    t.id === id ? { ...t, done: !t.done } : t
  ));
};
```

toggleDone 함수 구현

할 일 항목의 완료 상태(done)를 변경하는 함수

- 1 특정 id 항목 찾기
- 2 해당 항목의 done 값 반전 (true ↔ false)
- 3 불변성(immutability) 유지하며 상태 갱신

현재 todos 배열



map으로 모든 항목 순회



id 일치 항목만 done 상태 반전



새 배열로 상태 갱신

```
const toggleDone = (id) => {
  setTodos(todos.map(todo => {
    // id가 일치하는 항목만 done 값 반전
    if (todo.id === id) {
      return {
        ...todo, // 기존 속성 유지
        done: !todo.done // done 값만 반전
      };
    }
    return todo; // 다른 항목은 변경 없음
  }));
};

// 삼항 연산자 사용 간결한 버전
const toggleDone = (id) => {
  setTodos(todos.map(todo =>
    todo.id === id ? { ...todo, done: !todo.done }
    : todo
  ));
};
```

조건부 스타일 적용 (완료 표시)



인라인 스타일로 조건부 적용

삼항 연산자를 사용한 조건부 스타일링

완료 항목에 줄긋기(line-through) 효과 적용



객체 표기법 스타일

React의 인라인 스타일은 객체 형태로 지정

카멜케이스(camelCase) 속성명 사용



CSS 클래스 방식으로도 가능

조건부로 className 추가/제거하는 방식도 사용 가능

클래스명: `{` ${t.done ? "todo-done" : ""}`}`

```
function TodoItem({todo}) {  
  return ( <li style={{ textDecoration: todo.done  
    ? "line-through" : "none", color: todo.done ?  
    "#868e96" : "#212529" }} > {todo.text} </li> );  
}
```

☒ 리액트 기초 공부하기

☐ 투두리스트 앱 만들기

☒ 상태관리 이해하기

할 일 삭제 기능



삭제 버튼 UI 구현

각 할 일 항목마다 삭제 버튼을 추가하여 사용자가 완료된 작업이나 불필요한 항목을 제거할 수 있도록 구현



onClick 이벤트 핸들러

삭제 버튼 클릭 시 해당 항목의 id를 전달하여 deleteTodo 함수 호출

☒ 리액트 Hooks 공부하기

삭제

☐ To-do List 앱 완성하기

삭제

```
// 리스트 아이템에 삭제 버튼 추가
{todos.map((t) => (
  <li key={t.id}>
    <input
      type="checkbox"
      checked={t.done}
      onChange={() => toggleDone(t.id)}
    />
    {t.text}
    <button
      onClick={() => deleteTodo(t.id)}
    >
      삭제
    </button>
  </li>
))}
```


deleteTodo 함수 세부 구현

🔽 filter 메서드 활용

특정 id와 일치하지 않는 항목만 필터링하여 새 배열 생성
원본 배열을 직접 변경하지 않고 새로운 배열 반환

🔗 불변성(Immutability) 원칙

기존 상태를 직접 수정하지 않고 새로운 상태 객체 생성
React의 상태 관리와 렌더링 최적화의 핵심 원칙

불변성이 중요한 이유

1. 변경 사항 추적 용이
2. 리렌더링 최적화
3. 이전 상태 참조 가능 (시간 여행 디버깅)

```
const deleteTodo = (id) => { // filter: 조건에 맞는 항목으로 새 배열 생성 setTodos(  
  todos.filter(todo => todo.id !== id) ); // ❌  
  이렇게 하면 안됨 (불변성 위반) // const index =  
  todos.findIndex(t => t.id === id); //  
  todos.splice(index, 1); // setTodos(todos); //  
  원본 배열 변경 }; // 사용 예: <button onClick={()  
=> deleteTodo(todo.id)}> 삭제 </button>
```

남은 할 일 개수 표시

실시간 카운팅 구현

할 일 추가/완료/삭제 시 자동으로 남은 할 일 개수 업데이트

filter() 메소드 활용

JavaScript의 filter()를 사용해 완료되지 않은 할 일만 필터링

사용자 경험(UX) 향상

남은 작업량을 한눈에 파악하여 사용자 효율성 증가

```
// JSX 렌더링 부분에 추가
<ul> {todos.map((t) => ( /* 할 일 항목들... */
))} </ul> const remainingCount = todos.filter(t
=> !t.done).length; <p>남은 할 일:
{remainingCount}</p>
```

전체 할 일: 5개 / 남은 할 일 **3**

- ☒ React 기초 학습 *완료*
- ☒ To-Do List 앱 완성 *완료*
- ☐ 발표자료 준비
- ☐ CSS 스타일링 개선
- ☐ 단위 테스트 작성

참신한 팁: 이모지 활용

이모지로 목록에 생기 붙여넣기

할 일 앞에 적절한 이모지를 추가하면 시각적 구분이 쉽고 목록에 개성과 재미를 더할 수 있습니다.

할 일 유형별 이모지 사용하기

할 일의 성격에 맞는 이모지를 사용해 카테고리를 직관적으로 구분하세요. 학습은 📖, 운동은 💪, 업무는 💼 등으로 표현할 수 있습니다.

구현 방법

할 일 객체에 이모지 필드를 추가하거나, 할 일 텍스트 앞에 직접 이모지를 붙여 표시할 수 있습니다.

이모지가 있는 할 일 목록

📖 리액트 Hooks 공부하기

💪 30분 운동하기

🍴 건강한 저녁 준비하기

✉ 이메일 회신하기

```
// 이모지가 포함된 할 일 객체 예시
const todos = [ { id: 1, text: "📖 리액트 Hooks 공부하기", done: false }, { id: 2, text: "💪 30분 운동하기", done: false }, { id: 3, text: "🍴 건강한 저녁 준비하기", done: false } ];
```

학습 & 업무



생활 & 건강



참신한 팁: 우선순위 태그

우선순위 필드 추가하기

할 일 데이터 구조에 priority 필드를 추가하여 작업의 중요도를 표현할 수 있습니다. 우선순위로 시각적 차이를 두면 사용자가 중요한 할 일을 쉽게 식별할 수 있습니다.

색상 코드로 시각화

중요도에 따라 다른 색상을 적용하여 직관적인 UI를 만듭니다.

- **High (높음)**: 빨간색 - 긴급하고 중요한 일
- **Medium (중간)**: 주황색/노란색 - 중요하지만 긴급하지 않은 일
- **Low (낮음)**: 회색 - 나중에 해도 되는 일

우선순위 기반 필터링

우선순위로 할 일을 정렬하거나 필터링하여 사용자가 현재 상황에 맞는 작업에 집중할 수 있도록 도와줍니다.

```
// 우선순위 필드가 포함된 할 일 객체
const newTodo = {
  id: Date.now(),
  text: "보고서 작성하기",
  done: false,
  priority: "high" // "high", "medium", "low"
};

// 조건부 스타일링 예시
<li
  className={`todo-item ${todo.priority}-priority`}
  key={todo.id}
>
  {todo.text}
  <span className={`${todo.priority}-badge`} >
    {todo.priority}
  </span>
</li>
```

! 보고서 작성하기

high

→ 이메일 답장하기

medium

✓ 문서 정리하기

low

참신한 팁: 오늘 날짜 표시



사용자 경험 향상

오늘의 날짜를 앱 헤더에 표시하면 사용자에게 시간적 맥락을 제공하고 현재성을 부여하여 매일 새로운 리스트를 작성하는 느낌을 줍니다.



간단한 JavaScript Date 객체 활용

new Date()로 현재 날짜와 시간 객체를 생성하고, toLocaleDateString() 메소드를 사용하여 사용자 환경에 맞는 날짜 형식으로 표시합니다.



지역화 지원

toLocaleDateString()에 언어 코드를 전달하여 다국어 지원이 가능합니다. 옵션으로 날짜 형식을 세밀하게 조정할 수도 있습니다.

참고: 'ko-KR', 'en-US' 등의 로케일 코드를 사용하여 다양한 국가별 형식 지원 가능

```
{/* 헤더에 오늘 날짜 표시하기 */}
<h2>{new Date().toLocaleDateString()} To-do
List</h2>

{/* 옵션을 통한 날짜 형식 커스터마이징 */}
<h2>{new Date().toLocaleDateString('ko-KR', {
  year: 'numeric',
  month: 'long',
  day: 'numeric',
  weekday: 'long'
})}</h2>
```

기본 형식

2025. 9. 23.

상세 형식

2025년 9월 23일 화요일

할 일 목록

2025. 9. 23.

실제 앱 헤더 모습 예시

참신한 팁: 랜덤 배경색 적용



시각적 다양성 추가하기

항목마다 다른 배경색을 적용하여 할 일 목록에 시각적 구분을 제공하고 사용자 경험을 향상시킬 수 있습니다. 이는 항목 구분이 쉽고 좀 더 흥미로운 인터페이스를 만듭니다.



배경색 적용 방법

미리 정의된 색상 배열을 만들고, 각 항목의 고유 ID에 모듈로 연산자 (%)를 적용하여 색상 배열에서 색상을 선택합니다. 이를 통해 항목마다 예측 가능하면서도 다양한 색상을 적용할 수 있습니다.



접근성 고려하기

배경색을 선택할 때는 텍스트와의 충분한 대비를 유지하고, 너무 밝거나 눈에 피로를 주는 색상은 피하세요. 파스텔톤이나 부드러운 색상이 가독성과 사용자 경험에 좋습니다.

팁: 같은 색상의 다양한 명도를 사용하면 조화로운 디자인을 만들 수 있습니다.

```
// 색상 배열 정의
const colors = [
  "#e7f5ff", // 연한 파랑
  "#fff9db", // 연한 노랑
  "#f4fce3", // 연한 초록
  "#fff5f5"  // 연한 빨강
];

// 스타일 적용 방법
<li
  key={t.id}
  style={{ backgroundColor: colors[t.id %
colors.length] }}
>
  {t.text}
</li>
```

할 일 목록

☐ 리액트 Hooks 학습하기

삭제

☐ 투두리스트 UI 디자인하기

삭제

☒ 개발환경 세팅하기

삭제

☐ 최종 프로젝트 제출하기

삭제

참신한 팁: 자동 저장 기능

useEffect + localStorage로 자동 저장

할 일 목록이 변경될 때마다 브라우저의 localStorage에 자동으로 저장하여, 새로고침해도 데이터가 유지됩니다.

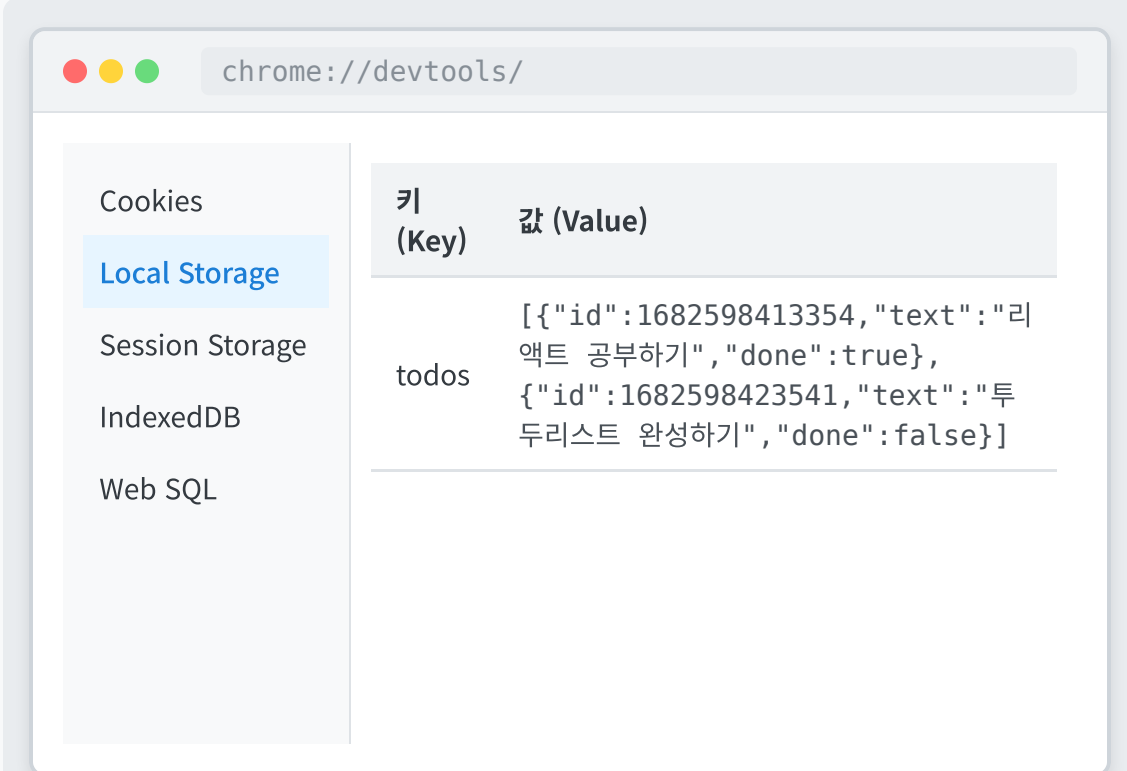
```
// 할 일 목록 초기화 (저장된 데이터 불러오기)
const [todos, setTodos] = useState(() => {
  const saved = localStorage.getItem("todos");
  return saved ? JSON.parse(saved) : [];
});

// todos가 변경될 때마다 자동 저장
useEffect(() => {
  localStorage.setItem("todos",
    JSON.stringify(todos));
}, [todos]);
```

자동 저장의 장점

사용자 경험 향상: 페이지 새로고침이나 브라우저를 닫았다 열어도 작업 내용이 그대로 유지됩니다.

참고: localStorage는 문자열만 저장할 수 있어 객체나 배열은 **JSON.stringify()**로 변환하고, 불러올 땐 **JSON.parse()**로 다시 변환해야 합니다.



The screenshot shows the Chrome DevTools interface with the 'Local Storage' tab selected. A table displays the stored data for the 'todos' key.

키 (Key)	값 (Value)
todos	[{"id":1682598413354,"text":"리액트 공부하기","done":true}, {"id":1682598423541,"text":"투두리스트 완성하기","done":false}]

확인 방법: 브라우저에서 F12 또는 개발자 도구 → Application 탭 → Storage → Local Storage

전체 코드 1/2 (TodoApp.jsx)



필요한 모듈 import

React Hooks(useState, useEffect) 불러오기



상태(state) 관리

입력값(text), 할 일 목록(todos) 상태 관리
localStorage에서 초기 데이터 로드



자동 저장 구현

useEffect로 todos 변경 시마다
localStorage에 자동 저장



이벤트 핸들러 함수

추가(handleSubmit), 완료 토글(toggleDone), 삭제
(deleteTodo) 기능 구현

```
import { useState, useEffect } from "react";

export default function TodoApp() {
  // 입력값과 할 일 목록 상태 관리
  const [text, setText] = useState("");
  const [todos, setTodos] = useState(() => {
    const saved = localStorage.getItem("todos");
    return saved ? JSON.parse(saved) : [];
  });

  // localStorage 자동 저장
  useEffect(() => {
    localStorage.setItem("todos", JSON.stringify(todos));
  }, [todos]);

  // 할 일 추가 함수
  const handleSubmit = (e) => {
    e.preventDefault();
    const trimmed = text.trim();
    if (!trimmed) return;
    setTodos([...todos, { id: Date.now(), text: trimmed, done: false }]);
    setText("");
  };

  // 완료 상태 토글 함수
  const toggleDone = (id) => {
    setTodos(todos.map(t =>
      t.id === id ? { ...t, done: !t.done } : t
    ));
  };

  // 할 일 삭제 함수
  const deleteTodo = (id) => {
    setTodos(todos.filter(t => t.id !== id));
  };
}
```


전체 코드 2/2 (TodoApp.jsx)

```
// JSX 렌더링 부분
return (
  <div className="card">
    <h2>{new Date().toLocaleDateString()} To-do List</h2>

    <form onSubmit={handleSubmit}>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
        placeholder="할 일을 입력하세요"
      />
      <button type="submit">추가</button>
    </form>

    <ul>
      {todos.map((t) => (
        <li key={t.id} style={{ textDecoration: t.done ? "line-through"
: "none" }}>
          <input type="checkbox" checked={t.done} onChange={() =>
toggleDone(t.id)} />
          {t.text}
          <button onClick={() => deleteTodo(t.id)}>삭제</button>
        </li>
      ))}
    </ul>

    <p>남은 할 일: {todos.filter(t => !t.done).length}</p>
  </div>
);
```

JSX 렌더링 부분 설명

-  **날짜 표시** - 현재 날짜를 동적으로 표시하여 매일 새로운 느낌의 To-do List 제공
-  **map 함수** - 배열의 각 항목을 JSX로 변환하여 목록 렌더링
-  **조건부 스타일** - 완료된 항목에 취소선(line-through) 적용
-  **남은 할 일 개수** - filter로 미완료 항목 실시간 카운팅

App.js 연결 및 구조



TodoApp 컴포넌트 불러오기

import 구문으로 TodoApp 컴포넌트를 App.js에 가져옵니다.



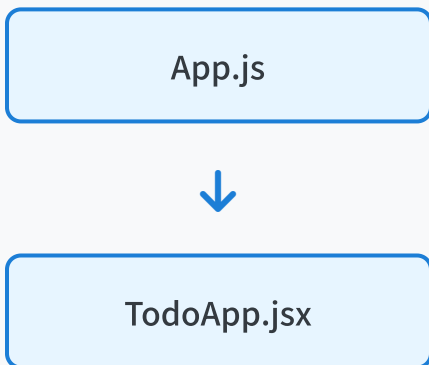
App 컴포넌트 내에서 사용

JSX에 TodoApp 컴포넌트를 배치하여 화면에 렌더링합니다.



컴포넌트 구조

App이 TodoApp을 포함하는 계층 구조로 구성됩니다.



```
// App.js
import React from 'react';
import TodoApp from '../components/TodoApp';

function App() {
  return (
    <div className="container">
      <header>
        <h1>나의 할 일 관리 앱</h1>
      </header>
      <main>
        <TodoApp />
      </main>
    </div>
  );
}

export default App;
```

CSS 샘플 및 스타일링



카드 스타일

앱의 메인 컨테이너에 적용할 카드형 디자인



할 일 항목 스타일

리스트 항목 구분 및 여백 관리



버튼 스타일 및 간격

사용자 상호작용 요소 디자인



반응형 레이아웃

다양한 화면에서 적절하게 표시

```
.card {
  max-width: 500px;
  margin: 0 auto;
  padding: 16px;
  border: 1px solid #ddd;
  border-radius: 12px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}

.todo-item {
  display: flex;
  align-items: center;
  margin-bottom: 8px;
  padding: 8px 0;
}

button {
  margin-left: 8px;
  padding: 4px 8px;
  border-radius: 4px;
  background: #4dabf7;
  color: white;
  border: none;
  cursor: pointer;
}
```

확장 아이디어 & 응용팁

▼ 필터 버튼 추가하기

할 일 항목을 '전체/활성/완료' 상태로 필터링하는 버튼을 추가하여 사용자가 원하는 항목만 볼 수 있도록 구현합니다.

⬆️ 완료 항목 정렬 기능

체크된 항목은 목록의 맨 아래로 자동 이동하도록 하여 남은 작업에 집중할 수 있게 합니다. 우선순위별 정렬도 구현 가능합니다.

📱 모바일 UI 최적화

반응형 디자인과 터치 인터페이스를 고려하여 모바일에서도 사용하기 좋은 UI로 개선합니다. 화면 크기에 따라 레이아웃 조정이 필요합니다.

🔗 할 일 공유 기능

카카오톡, 이메일 등으로 할 일 목록을 공유할 수 있는 기능을 추가하여 협업이나 리마인더 용도로 활용할 수 있습니다.

모두 보기

활성 항목

완료 항목

```
// 필터 상태 관리
const [filter, setFilter] =
  useState('all');

// 필터링된 할 일 목록
const filteredTodos = todos.filter(todo =>
{
  if (filter === 'all') return true;
  if (filter === 'active') return
    !todo.done;
  if (filter === 'completed') return
    todo.done;
});
```

오늘의 할 일

- ☒ 장보기 
- ☐ 리액트 공부하기 
- ☐ 운동 30분 

완료된 할 일

- ☒ 장보기 



앱 실행 화면 예시



완성된 To-do List 앱

지금까지 배운 모든 기능이 통합된 To-do List 앱의 실제 동작 화면입니다. 사용자 입력, 할 일 관리, 상태 표시 기능들을 확인해보세요.



핵심 기능 시연

할 일 추가, 체크박스로 완료 표시, 삭제, 남은 할 일 갯수 표시 등 주요 기능을 확인할 수 있습니다.



localStorage 자동 저장

브라우저를 새로고침해도 작성한 할 일 목록이 유지됩니다. 개발자 도구(F12)의 Application 탭에서 localStorage 저장 내용을 확인해보세요.

실습 과제: 앱에 이모지, 우선순위 기능 등 학습한 참신한 팁들을 추가해보세요!



1

입력

새로운 할 일 작성

2

추가

목록에 항목 반영

3

완료

체크박스 클릭

4

삭제

불필요 항목 제거

5

저장

자동 로컬 저장

학습 내용 정리

이벤트 처리

- 1 `onChange` : 입력값을 state로 관리
- 2 `onSubmit` : 폼 제출 제어 (e.preventDefault 활용)
- 3 Controlled Component 패턴 활용

UI 처리

- 1 조건부 스타일링: `t.done ? "line-through" : "none"`
- 2 체크박스 상태 연동: `checked={t.done}`
- 3 CSS 클래스 동적 적용

리스트 렌더링

- 1 `map()` : 배열을 컴포넌트 리스트로 변환
- 2 `key` prop: 리액트의 효율적 DOM 업데이트
- 3 `filter()` : 항목 삭제 및 남은 할 일 계산

실용적인 팁

- 1 이모지와 우선순위 태그로 시각적 구분
- 2 `localStorage` : 자동 저장 구현
- 3 `toLocaleDateString()` : 날짜 표시
- 4 `useEffect` : 데이터 지속성 관리

실전 퀴즈



Q1: onSubmit에서 e.preventDefault()가 필요한 이유?

힌트

폼의 기본 동작과 React의 상태 관리 방식을 생각해보세요. `preventDefault()` 메서드가 없다면 페이지에 어떤 일이 발생하나요?



Q2: key prop을 반드시 써야 하는 이유?

힌트

React의 가상 DOM 업데이트 알고리즘과 요소 식별 방식에 대해 생각해보세요. `key` 가 없다면 리스트 항목 변경 시 어떤 문제가 발생할까요?



Q3: localStorage를 쓰면 얻는 장점은?

힌트

브라우저가 새로고침되거나 닫힌 후에도 데이터 유지가 필요한 경우를 생각해보세요. `localStorage` 와 `useEffect` 를 어떻게 결합하여 사용했나요?

마무리 및 참고자료

✓ 오늘 배운 내용

- 1 To-do List 기본 기능 구현 완료
- 2 React의 핵심 개념 (Controlled Form, 상태관리) 실습
- 3 이벤트 핸들링, 목록 렌더링, 조건부 렌더링 활용

➔ 다음 주차 예고

- 🌐 외부 API 연동 실습
- ⚙️ 날씨 정보 앱 만들기
- 🎬 영화 정보 앱 실습

📖 학습 참고자료

- 🔗 React 공식 문서: <https://react.dev/learn>
- 🔗 MDN Web Docs – Event Handling:
<https://developer.mozilla.org/ko/docs/Web/Events>
- 🔗 Velopert 블로그, "React로 To-do List 만들기"

💡 추가 학습 아이디어

- 📄 TypeScript로 타입 안전성 추가하기
- 🗄️ 서버 연동으로 영구 저장소 구현
- 📱 React Native로 모바일 앱 전환